

Classification Problem:

Implementing the Perceptron algorithm For finding the
Weights of a Linear Discriminant function
(Equivalently: Linear Decision Boundary)
Between Two Classes

Suppose, We are given the Following Six Patterns (also called:
Instances / Data Points).

$\omega_1 = (1, 1), (1, -3), (4, 5)$

$\omega_2 = (2, 2.5), (0, 2), (2, 3)$

Each Pattern is represented as (x, y) where x and y are its Two
measured Features.

The first Three Patterns belong to Class 1 (shown as ω_1),
while the Last three Patterns belong to Class 2 (ω_2)

Objective:

We have to find a Linear Decision Boundary between These Two
Classes !

Related Terms:

(A data point or Pattern is a vector of measured features in the feature space)

A *pattern* is a vector of measured *features*: *Each pattern x can be thought as a point in the multi-dimensional (i.e., n -dimensional) feature space*

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The classification problem is a machine learning problem where the Goal is to ...

Goal is to assign each unclassified pattern $\mathbf{x}$ to one of the predefined set of classes $\{\omega_i\}$.

Feature Spaces

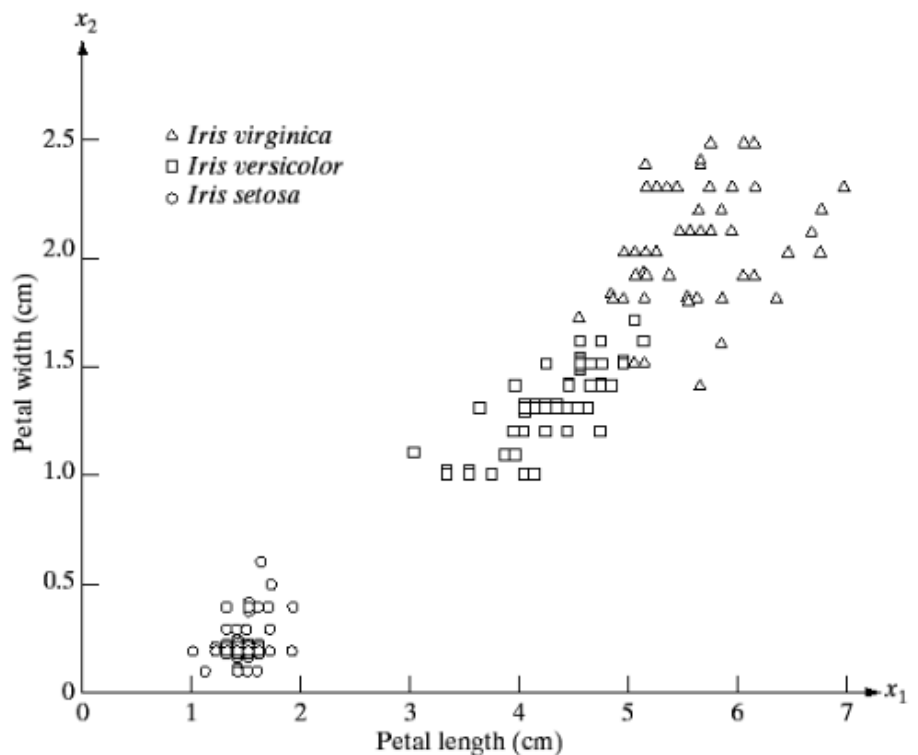
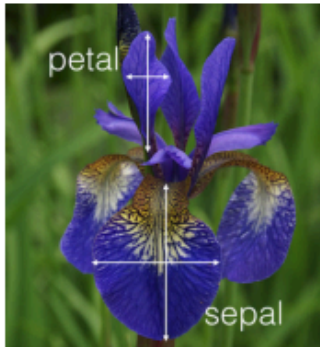
Can think of each pattern as a point in a **feature space**:

FIGURE 12.1

Three types of iris flowers described by two measurements.

Feature₁ along x-axis (x_1)
= Petal Width

Feature₂ along y-axis (x_2)
= Petal Length



Linear Decision Boundary: The Decision Boundary between **class 1** and **class 2** is the Straight Line (i.e., set of Points) where both the classes Are equal likely (i.e., **Discriminant Function value $g(x) = 0$** for all the points x on that Line). Here, pattern x has two components x_1 and x_2 , that is, $\mathbf{x} = [x_1 \ x_2]^T$

For All the Patterns (data points) of One Class, **$g(x) > 0$**

For All the Patterns (data points) of The Other Class, **$g(x) < 0$**

On Decision Boundary Line, $g(\mathbf{x}) = 0$

Decision Boundaries

If we partition the feature space according to the nearest prototype of each class, we create **decision boundaries**: *Prototype of each class,*

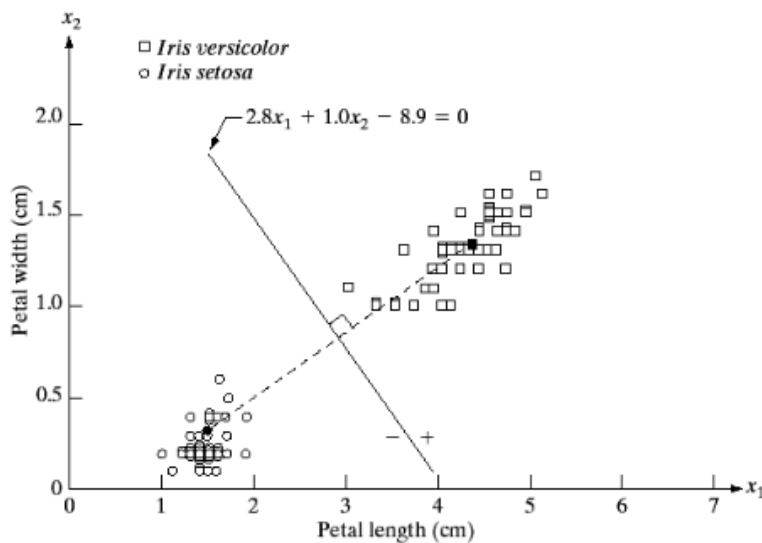


FIGURE 12.6
Decision boundary of minimum distance classifier for the classes of *Iris versicolor* and *Iris setosa*. The dark dot and square are the means.

If we partition the feature space according to the nearest *Prototype of each class*, we create **decision boundaries**:

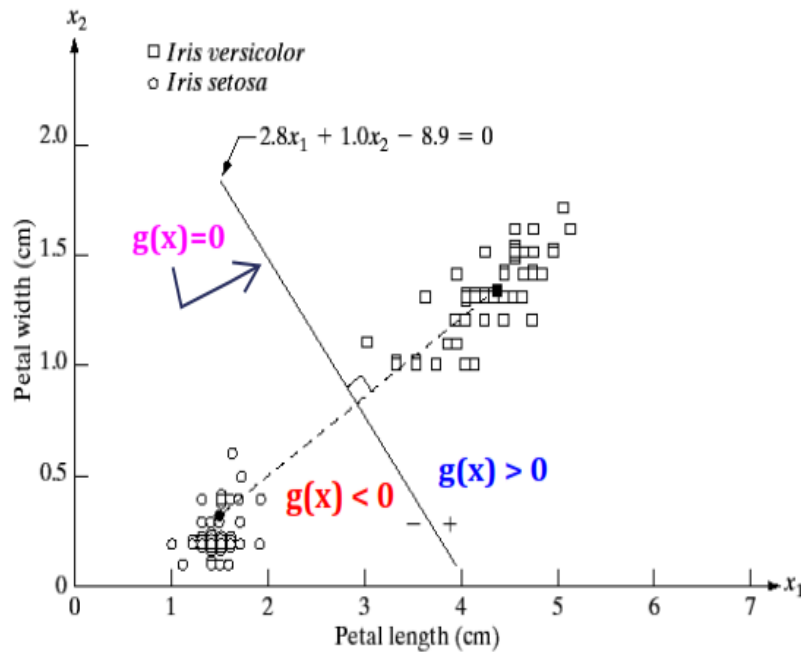


FIGURE 12.6
Decision boundary of minimum distance classifier for the classes of *Iris versicolor* and *Iris setosa*. The dark dot and square are the means.

Here, The **Equation of the Decision Boundary $g(x)$** is

$$g(x) = g(x_1, x_2) = 2.8 x_1 + 1 x_2 - 8.9 \quad \{ \text{the Pattern } x \text{ has two features } x_1, x_2 \}$$

or, More Formally: $x = [x_1 \ x_2]$

As the decision boundary is a Straight Line, it has general straight line equation:

$$ax + by + c = 0$$

or, where two dimensions are x_1, x_2 $\square$ $ax_1 + bx_2 + c = 0$

$$[a \ b] * \begin{bmatrix} x_1 & x_2 \end{bmatrix} + \text{constant } c = 0$$

Equivalently:

Discriminant Function $g(\mathbf{x})$: $\mathbf{w}^T \mathbf{x} + w_0 = 0 \quad \dots \dots \dots$
(1)

Another way to represent above (which we will need later):

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = [\mathbf{w}^T \quad w_0] \begin{bmatrix} x & 1 \end{bmatrix} = [\mathbf{w}^T \quad w_0] \begin{bmatrix} x & x_0 \end{bmatrix}$$

*** Note, $x_0 = 1$ always. Because x_0 is the Last component of $\mathbf{x}$ vector is set to 1 always to keep w_0 free/independent constant, not dependent on any dimension, like the constant 'c' in the straight line eq. $ax+by+c=0$, this constant 'c' acts as y intercept or SHIFT of the line or hyperplane in multi dimensions ... ****

$\mathbf{w}^T$ is the row-vector (i.e., column transpose) : $\mathbf{w}^T = [\mathbf{a} \quad \mathbf{b}]$
 $\mathbf{x}$ is the feature vector $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}$

- $\mathbf{w}^T$ is always vector with **SAME number of components** as the no. of **Feature components** in $\mathbf{x}$.
- For example, if $\mathbf{x} = [x_1 \quad x_2]^T$, then $\mathbf{w} = [w_1 \quad w_2]^T$.
If $\mathbf{x}^T$ has three features $[x_1 \quad x_2 \quad x_3]$, then $\mathbf{w}^T = [w_1 \quad w_2 \quad w_3]$
- w_0 is the constant term (c) in the Straight line equation (the y-intercept c in the form $y = mx + c$)
- To Find the decision Boundary, we have to find the values of $\mathbf{w}^T$ and w_0

*** Perceptron Algorithm Steps Summary ***

Step 1: High Dimensional Projection

- The Training Patterns May NOT be **Linearly separable**.
- It is often found that A Group of Points which are **NOT Linearly Separable** in Low Dimensional Space May Become easily **Linearly Separable in Higher Dimensional Space**.
- This is why We will project All the 2-D (x, y) Patterns/Data Points into Higher dimensions by Using a Function, called **ϕ (PHI) function**.
- For example, If we want to Project the 2-D points into 6-D space, we may use the following ϕ function:

$$y = \phi(x) = [x_1^2 \quad x_2^2 \quad x_1 * x_2 \quad x_1 \quad x_2 \quad 1]^T \quad \dots \quad (2)$$

The Last Component of y always must be = 1. Otherwise, it will change the **w_0 (constant term)** in the Discriminant functions equation (eq. 1, 3). If it is 1, then it will not change the w_0 term in eq. 3 when multiplied with w_0 and thus at least one weight-component (w_0) remains Constant (not dependent on any

feature component) which is similar to the constant term of a straight line (two dimensional 2D case) or hyper plane (high dimensional case). More precisely,

$$\text{Discriminant Function } g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = [\mathbf{w}^T \quad w_0] [x \ 1] \quad \dots \quad (3)$$

The vector $[\mathbf{w}^T \quad w_0]$ is called “weight vector” and the vector $[x \ 1]$ is called “Augmented Feature Vector” because of the ‘1’ augmented as the end component of $\mathbf{x}$.

If we express the Eq. (3) in Sum of Products term, it becomes:

$$g(\mathbf{x}) = \sum_{i=0}^n w_i x_i \text{ where } x_0 = 1, \text{ which is multiplied with } w_0 \text{ to produce a constant, which is necessary because } g(\mathbf{x}) \text{ must be a decision LINE (or, hyper plane) which has the general form}$$

$$y = mx + c$$

This constant c is produced by $w_0 * x_0 = w_0$, since $x_0 = \text{always } 1$ (constant)

However, after the High dimensional projection, the 2-D $\mathbf{x}$ -vector now becomes high dimensional (6-D) $\mathbf{y}$ -vector.

Since the $\mathbf{y}$ -vector has 6 components, the weight vector $[\mathbf{w}^T \quad w_0]$ will also have 6 components $\mathbf{a}^T = [w_5 \ w_4 \ w_3 \ w_2 \ w_1 \ w_0]$.

This $\mathbf{y}$ is called “High Dimensional Augmented Feature Vector” and

$a^T = [w_5 \ w_4 \ w_3 \ w_2 \ w_1 \ w_0]$ is called “High Dimensional augmented weight vector”

Now, the Discriminant function uses the high-Dimensional vector y :

$$g(y) = a^T y = [w_5 \ w_4 \ w_3 \ w_2 \ w_1 \ w_0] * [x_1^2 \quad x_2^2 \quad x_1 * x_2 \quad x_1 \quad x_2 \quad 1]^T$$

If $g(y) > 0$ then

$y \in \omega_1$ [i.e., y belongs to Class 1]

If $g(y) < 0$ then

$y \in \omega_2$ [i.e., y belongs to Class 2]

Generating the high dimensional sample points by the following formula:

$$y = \phi(x) = [x_1^2 \quad x_2^2 \quad x_1 * x_2 \quad x_1 \quad x_2 \quad 1]^T$$

$(x_1, y_1) = (1, 1)$	y_{11}	1 1 1 1 1 1
$(x_2, y_2) = (1, -3)$	y_{12}	1 9 -3 1 -3 1
$(x_3, y_3) = (4, 5)$	y_{13}	16 25 20 4 5 1
$(x_4, y_4) = (2, 2.5)$	$-y_{21}$	-4 -6.25 -5 -2 -2.5 -1
$(x_5, y_5) = (0, 2)$	$-y_{22}$	0 -4 0 0 -2 -1
$(x_6, y_6) = (2, 3)$	$-y_{23}$	-4 -9 -6 -2 -3 -1

But, why we used Negated (Reflected) values for the patterns in Class 2?

The answer is in the following point: Class Normalization

Step 2: Class Normalization

After Producing y and a , where

y : High Dimensional Augmented Feature Vector and

a : High Dimensional augmented weight vector

Now, the Discriminant function becomes

$$g(y) = a^T y = [w_5 \ w_4 \ w_3 \ w_2 \ w_1 \ w_0] * \begin{bmatrix} x_1^2 & x_2^2 & x_1 * x_2 & x_1 & x_2 & 1 \end{bmatrix}$$

At this point, The **Classification Decision Program** looks like this:

If $g(y) > 0$ then

$y \in \omega_1$

If $g(y) < 0$ then

$y \in \omega_2$

Notice: We have to Take care of Two conditions: $g(y) > 0$ and $g(y) < 0$

But the 2nd condition $g(y) < 0$ can be replaced by $g(-y) > 0$

- Thus, If we **Replace All the Patterns of Class 2 by their Reflected Prototypes** (i.e., replace y by $-y$, that is, **SET** $y = -y$ for all $y \in \omega_2$), then we have to take care of **ONLY 1 condition**:

$g(y) > 0$. This makes the Perceptron Training Algorithm Much Simpler, with only One condition, which is as follows:

Adjust the weight vector Again and Again
Until $g(y) > 0$ for All the Training Patterns (i.e., all the y 's)

Thus Replacing all Patterns of a particular class by their Reflected Prototypes is called “Normalizing” that particular class.

For example, In the previous Table, we have Normalized the Class 2.

Step 3: Perceptron Algorithm Initialization

- Learning Rate = α in the range $[0, 1]$. That is, $0 < \alpha \leq 1$
- If we Set the Learning Rate $\alpha = 0$, NO weight update will take place EVER. (see Eq. 4)
- In case $\alpha > 1$, The weight updates May become Too Large (Step Size Too Large), which will probably make the algorithm Unstable!
- Suitable Value of α for many Problems ~ 0.5
- A more appropriate value for α is found to be $1/n$, where n is the no. of training patterns (i.e., 6 in our example).

Thus, $\alpha = 1/6 = 0.17$

=====

- The initial weight vector components are set arbitrary.
- In our example we have set it to all 1's (i.e., [1 1 1 1 1 1]).
- A better approach is to set the weight values to **Random Small values, such as Uniform_Random from [-1, 1]**.
- However, DON'T set all the weights to 0's, because it will stop any training.

Perceptron Weight Update Rule:

Use Perceptron Algorithm (one at a time) for finding the weight-coefficients of the discriminant function (i.e., values of w) boundary for your linear classifier
Here α is the learning rate and $0 < \alpha \leq 1$.

$$\begin{aligned} \underline{\mathbf{w}}(\mathbf{i}+1) &\rightarrow \underline{\mathbf{w}}(\mathbf{i}) + \alpha \tilde{\mathbf{y}}_m^{(k)} && \text{if } \underline{\mathbf{w}}^T(\mathbf{i}) \tilde{\mathbf{y}}_m^{(k)} \leq 0 \\ &\rightarrow \text{(i.e., if } \tilde{\mathbf{y}}_m^{(k)} \text{ is misclassified)} \\ \underline{\mathbf{w}}(\mathbf{i}+1) &\rightarrow \underline{\mathbf{w}}(\mathbf{i}) && \text{if } \underline{\mathbf{w}}^T(\mathbf{i}) \tilde{\mathbf{y}}_m^{(k)} > 0 \end{aligned} \quad \dots (4)$$

This means:

If $g(y) > 0$

then $\hat{y}$ is correctly classified. So, **Keep the weights Unchanged**

But if $g(y) < 0$

then y is misclassified. We call this y vector as $\tilde{y}_{-m}^{(k)}$. The weight vector is changed by **Adding a portion (α) of this misclassified y vector with the existing weight vector**, i.e.,

$$w(i+1) = w(i) + \alpha \tilde{y}_{-m}^{(k)} \quad \dots (4b)$$

Note: The weight vector $w(i)$ and the High Dimensional feature vector y Have **SAME** no. of components

Two Approaches for Weight Update:
One at a Time
and
Many at a Time

In the **One at a Time** weight update strategy, whenever any y is found to be **misclassified** (i.e., $g(y) < 0$), the **weight values are updated immediately** by using Eq. (4)

In **Many at a Time** weight update approach, the weight values are updated **only Once AFTER each iteration** (i.e., **NOT WITHIN** the iteration), after processing all the y -vectors by the discriminant function $g(y)$. During each iteration, **the misclassified y -vectors** (i.e., the y vectors for which $g(y) < 0$) **are summed**, and at the end of

the iteration, the current weight values are **Updated by Adding a portion (α) of the Summed vector**. More precisely,

$$w(i+1) = w(i) + \alpha \sum y_{-m}^{(k)} \quad \dots (4)$$

Here, the sum is defined over all the misclassified y-vectors $\tilde{y}_{-m}^{(k)}$.

See the detailed example, as follows.

When to Stop (Convergence):

If During any **Complete iteration**, **None of the y-vectors are misclassified** (i.e., $g(y) > 0$ for all the y-vectors during an iteration) And hence **NONE of the weight values are changed** throughout the full iteration, then the Perceptron Algorithm is Converged and Stopped.

Usually the Algorithm requires a few hundreds, and sometimes even several thousands of iterations before convergence.

The no. of iterations depend on the Difficulty of the Problem (the number and complexity of the Patterns to be classified), Learning rate value α , The initial weight vector, The strategy used – One at a time or Many at a time and so on.

Usually Simpler Problems with Small number of Patterns, Random initial weight values and the Many-At-A-Time strategy require Smaller no. of iterations.

Demonstration of the Complete Perceptron Algorithm (Two Complete Iterations):

One at a Time Approach:

We have set the Learning Rate $\alpha = 0.8$

Initial weight vector $\omega = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$

So, $\omega^T = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$

Given,

$\omega_1 = (1, 1), (1, -3), (4, 5)$ and

$\omega_2 = (2, 2.5), (0, 2), (2, 3)$

Merging these two classes, we get $c = (1, 1), (1, -3), (4, 5) (2, 2.5), (0, 2), (2, 3)$

Generating the high dimensional sample points by the following formula:

$$y = [x_i^2 \quad y_i^2 \quad x_i * y_i \quad x_i \quad y_i \quad 1]^T$$

$(x_1, y_1) = (1, 1)$	y_{11}	1	1	1	1	1	1
-----------------------	----------	---	---	---	---	---	---

$(x_2, y_2) = (1, -3)$	y_{12}	1 9 -3 1 -3 1
$(x_3, y_3) = (4, 5)$	y_{13}	16 25 20 4 5 1
$(x_4, y_4) = (2, 2.5)$	(Reflected) y_{21}	-4 -6.25 -5 -2 -2.5 -1
$(x_5, y_5) = (0, 2)$	(Reflected) y_{22}	0 -4 0 0 -2 -1
$(x_6, y_6) = (2, 3)$	(Reflected) y_{23}	-4 -9 -6 -2 -3 -1

Iteration 1:

Weight Update Rule : if $\omega^T y < 0$ then

$$\omega = \omega + \alpha * y$$

Step 1:

$y_{11} = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$, so

$$\omega^T y = [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$= 1*1 + 1*1 + 1*1 + 1*1 + 1*1 + 1*1$$

$$= 1 + 1 + 1 + 1 + 1 + 1$$

$$= 6 > 0, \text{ So weights are kept unchanged !}$$

Step 2:

$y_{12} = [1 \ 9 \ -3 \ 1 \ -3 \ 1]$, so

$$\omega^T y = [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [1 \ 9 \ -3 \ 1 \ -3 \ 1]$$

$$= 1*1 + 1*1 + 1*(-3) + 1*1 + 1*(-3) + 1*1$$

$$= 1 + 1 - 3 + 1 - 3 + 1$$

$$= -2$$

Here, $\omega^T y = -2 < 0$ (Negative)

So the new ' ω ' is calculated as follows:

$$\omega = \omega + \alpha * y_{12}$$

$$= [1 \ 1 \ 1 \ 1 \ 1 \ 1] + 0.8 * [1 \ 9 \ -3 \ 1 \ -3 \ 1]$$

$$= [1.8 \ 8.2 \ -1.4 \ 1.8 \ -1.4 \ 1.8]$$

Step 3:

$$y_{13} = [16 \ 25 \ 20 \ 4 \ 5 \ 1], \text{ so}$$

$$\omega^T y = [1.8 \ 1.8 \ -1.4 \ 1.8 \ -1.4 \ 1.8] * [16 \ 25 \ 20 \ 4 \ 5 \ 1]$$

$$= 1.8*16 + 1.8*25 - 1.4*20 + 1.8*4 - 1.4*5 + 1.8*1$$

$$= 47.8 > 0, \text{ So weights are unchanged !}$$

Step 4:

$$y_{21} = [-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1], \text{ so}$$

$$\omega^T y = [1.8 \ 1.8 \ -1.4 \ 1.8 \ -1.4 \ 1.8] *$$

$$[-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1]$$

$$= 1.8 * (-4) + 1.8 * (-6.25) - 1.4 * (-5) + 1.8 * (-2) - 1.4 * (-2.5) + 1.8 * (-1)$$

$$= -13.35$$

Here, $\omega^T y = -13.35 < 0$ (Negative)

So the new ' ω ' is calculated as follows:

$$\begin{aligned}\omega &= \omega + \alpha * y_{21} \\ &= [1.8 \ 1.8 \ -1.4 \ 1.8 \ -1.4 \ 1.8] + 0.8 * \\ &\quad [-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1] \\ &= [-1.4 \ -3.2 \ 5.4 \ 0.2 \ -3.4 \ 1]\end{aligned}$$

Step 5:

$y_{22} = [0 \ -4 \ 0 \ 0 \ -2 \ -1]$, so

$$\omega^T y = [-1.4 \ -3.2 \ 5.4 \ 0.2 \ -3.4 \ 1] * [0 \ -4 \ 0 \ 0 \ -2 \ -1]$$

$$= -1.4 * 0 + (-3.2) * (-4) + (5.4) * 0 + 0.2 * 0 + (-3.4) * (-2) + 1 * (-1)$$

$$= 18.6 > 0, \text{ Hence NO adjustment of the weight values necessary !!!}$$

Step 6:

$y_{23} = [-4 \ -9 \ -6 \ -2 \ -3 \ -1]$, so

$$\begin{aligned}\omega^T y &= [-1.4 \ -3.2 \ 5.4 \ 0.2 \ -3.4 \ 1] * \\ &\quad [-4 \ -9 \ -6 \ -2 \ -3 \ -1]\end{aligned}$$

$$= (-1.4) * (-4) + (-3.2) * (-9) + (-5.4) * (-6) + (0.2) * (-2) + (-3.4) * (-3) + 1 * (-1)$$

$$= 75.6 > 0, \text{ Hence NO adjustment of the weight values necessary !!!}$$

Summary of the Steps:

y_{ij}	ω	$\omega^T y$
1 1 1 1 1 1	1 1 1 1 1 1	+ 6
1 9 -3 1 -3 1	1 1 1 1 1 1	- 2
16 25 20 4 5 1	1.8 8.2 -1.4 1.8 -1.4 1.8	47.8
-4 -6.25 -5 -2 -2.5 -1	1.8 1.8 -1.4 1.8 -1.4 1.8	-13.35
0 -4 0 0 -2 -1	-1.4 -3.2 -5.4 0.2 -3.4 1	18.6
-4 -9 -6 -2 -3 -1	-1.4 -3.2 -5.4 0.2 -3.4 1	75.6

If the last value 75.6 were negative, then you **MUST** show an additional update on w

Iteration 2:

Step 1:

$y_{11} = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$, so

$$\begin{aligned}
 \omega^T y &= [-1.4 \quad -3.2 \quad -5.4 \quad 0.2 \quad -3.4 \quad 1] * [1 \ 1 \ 1 \ 1 \ 1 \ 1] \\
 &= (-1.4) * 1 + (-3.2) * 1 + (-5.4) * 1 + (0.2) * 1 + (-3.4) * 1 + 1 * 1 \\
 &= -12.2
 \end{aligned}$$

Here, $\omega^T y = -12.2 < 0$ (Negative)

So the new ' ω ' is calculated as follows:

$$\begin{aligned}\omega &= \omega + \alpha * y_{11} \\ &= [-1.4 \quad -3.2 \quad -5.4 \quad 0.2 \quad -3.4 \quad 1] + 0.8 * [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] \\ &= [-0.6 \quad -2.4 \quad -4.6 \quad 1 \quad -2.6 \quad 1.8]\end{aligned}$$

Step 2:

$$\begin{aligned}y_{12} &= [1 \quad 1 \quad -3 \quad 1 \quad -3 \quad 1], \text{ so} \\ \omega^T y &= [-0.6 \quad -2.4 \quad -4.6 \quad 1 \quad -2.6 \quad 1.8] * [1 \quad 1 \quad -3 \quad 1 \quad -3 \quad 1] \\ &= (-0.6) * 1 + (-2.4) * 1 + (-4.6) * (-3) + 1 * 1 + (-2.6) * (-3) + 1.8 * 1 \\ &= 21.4\end{aligned}$$

Step 3:

$$\begin{aligned}y_{13} &= [16 \quad 25 \quad 20 \quad 4 \quad 5 \quad 1], \text{ so} \\ \omega^T y &= [-0.6 \quad -2.4 \quad -4.6 \quad 1 \quad -2.6 \quad 1.8] * [16 \quad 25 \quad 20 \quad 4 \quad 5 \quad 1] \\ &= (-0.6) * 16 + (-2.4) * 25 + (-4.6) * 20 + 1 * 4 + (-2.6) * 5 + 1.8 * 1 \\ &= -168.8\end{aligned}$$

Here, $\omega^T y = -168.8 < 0$ (Negative)

So the new ' ω ' is calculated as follows:

$$\begin{aligned}\omega &= \omega + \alpha * y_{13} \\ &= [-0.6 \quad -2.4 \quad -4.6 \quad 1 \quad -2.6 \quad 1.8] + 0.8 * [16 \quad 25 \quad 20 \quad 4 \quad 5 \quad 1] \\ &= [12.2 \quad 17.6 \quad 11.4 \quad 4.2 \quad 1.4 \quad 2.6]\end{aligned}$$

Step 4:

$$y_{21} = [-4 \quad -6.25 \quad -5 \quad -2 \quad -2.5 \quad -1], \text{ so}$$

$$\omega^T y = [12.2 \quad 17.6 \quad 11.4 \quad 4.2 \quad 1.4 \quad 2.6] * [-4 \quad -6.25 \quad -5 \quad -2 \quad -2.5 \quad -1]$$

$$= 12.2*(-4) + 17.6 * (-6.25) + 11.4 * (-5) + 4.2 * (-2) + 1.4 * (-2.5) + 2.6*(-1) \\ = -230.3$$

Here, $\omega^T y = -230.3 < 0$ (Negative)

So the new ' ω ' is calculated as follows:

$$\omega = \omega + \alpha * y_{21} \\ = [12.2 \quad 17.6 \quad 11.4 \quad 4.2 \quad 1.4 \quad 2.6] + 0.8 * [-4 \quad -6.25 \quad -5 \quad -2 \quad -2.5 \quad -1] \\ = [9 \quad 12.6 \quad 7.4 \quad 2.6 \quad -0.6 \quad 1.8]$$

Step 5:

$$y_{22} = -y_{21} = [0 \quad -4 \quad 0 \quad 0 \quad -2 \quad -1], \text{ so}$$

$$\omega^T y = [9 \quad 12.6 \quad 7.4 \quad 2.6 \quad -0.6 \quad 1.8] * [0 \quad -4 \quad 0 \quad 0 \quad -2 \quad -1]$$

$$= 9 * 0 + 12.6 * (-4) + 7.4 * 0 + 2.6 * 0 - 0.6 * (-2) + 1.8 * (-1)$$

$$= -51$$

Here, $\omega^T y = -51 < 0$ (Negative)

So the new ' ω ' is calculated as follows:

$$\begin{aligned}\omega &= \omega + \alpha * y_{22} \\ &= [9 \ 12.6 \ 7.4 \ 2.6 \ -0.6 \ 1.8] + 0.8 * [0 \ -4 \ 0 \ 0 \ -2 \ -1] \\ &= [9 \ 9.4 \ 7.4 \ 2.6 \ -2.2 \ 1]\end{aligned}$$

Step 6:

$$y_{23} = -y_{23} = [-4 \ -9 \ -6 \ -2 \ -3 \ -1], \text{ so}$$

$$\begin{aligned}\omega^T y &= [9 \ 9.4 \ 7.4 \ 2.6 \ -2.2 \ 1] * [-4 \ -9 \ -6 \ -2 \ -3 \ -1] \\ &= 9 * (-4) + 9.4 * (-9) + 7.4 * (-6) + 2.6 * (-2) + (-2.2) * (-3) + 1 * (-1) \\ &= -164.6\end{aligned}$$

Here, $\omega^T y = -164.6 < 0$ (Negative)

So the new ' ω ' is calculated as follows:

$$\omega = \omega + \alpha * y_{23}$$

(Calculate the weight values (ω), which will be used in the Next Iteration: Iteration 3)

Summary of the Steps:

y_{ij}	ω	$\omega^T y$
1 1 1 1 1 1	-1.4 -3.2 -5.4 0.2 -3.4 1	-12.2
1 1 -3 1 -3 1	-0.6 -2.4 -4.6 1 -2.6 1.8	21.4

16 25 20 4 5 1	-0.6 -2.4 -4.6 1 -2.6 1.8	-168.8
-4 -6.25 -5 -2 -2.5 -1	12.2 17.6 11.4 4.2 1.4 2.6	-230.3
0 -4 0 0 -2 -1	9 12.6 7.4 2.6 -0.6 1.8	-51
-4 -9 -6 -2 -3 -1	9.0 9.4 7.4 2.6 -2.2 1	-164.6

Repeat the iteration until all $\omega^T y$ are positive or the number of iterations are finished.

Many at a Time Approach

Given,

$$\omega_1 = (1, 1), (1, -3), (4, 5)$$

$$\omega_2 = (2, 2.5), (0, 2), (2, 3)$$

Merging these two classes, we get y.

$$c = (1, 1), (1, -1), (4, 5), (2, 2.5), (0, 2), (2, 3)$$

Generating the high dimensional sample points by the following formula:

$$y = [x_i^2 \quad y_i^2 \quad x_i * y_i \quad x_i \quad y_i \quad 1]$$

$(x_1, y_1) = (1, 1)$	y_{11}	1 1 1 1 1 1
$(x_2, y_2) = (1, -3)$	y_{12}	1 1 -3 1 -3 1
$(x_3, y_3) = (4, 5)$	y_{13}	16 25 20 4 5 1
$(x_4, y_4) = (2, 2.5)$	(Reflected) y_{21}	-4 -6.25 -5 -2 -2.5 -1
$(x_5, y_5) = (0, 2)$	(Reflected) y_{22}	0 -4 0 0 -2 -1

$(x_6, y_6) = (2, 3)$	(Reflected) y_{23}	-4	-9	-6	-2	-3	-1
-----------------------	----------------------	----	----	----	----	----	----

In many at a time approach for one iteration, only one ' ω ' is used. It is updated in the next iteration using the formula:

$$\omega = \omega + \alpha * \Sigma y_{ij}$$

Iteration-1 :

Set Learning Rate $\alpha = 0.8$

if $\omega^T y < 0$ then

$$\omega = \omega + \alpha * y_{ij}$$

Initially , $\omega = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$

So, $\omega^T = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$

Step 1:

$y_{11} = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$, so

$$\omega^T y = [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$= 1*1 + 1*1 + 1*1 + 1*1 + 1*1 + 1*1$$

$$= 1 + 1 + 1 + 1 + 1 + 1$$

$$= 6$$

Step 2:

$y_{12} = [1 \ 1 \ -3 \ 1 \ -3 \ 1]$, so

$$\omega^T y = [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [1 \ 1 \ -3 \ 1 \ -3 \ 1]$$

$$\begin{aligned}
&= 1*1 + 1*1 + 1*(-3) + 1*1 + 1*(-3) + 1*1 \\
&= 1 + 1 - 3 + 1 - 3 + 1 \\
&= -2
\end{aligned}$$

Step 3:

$$\begin{aligned}
y_{13} &= [16 \ 25 \ 20 \ 4 \ 5 \ 1], \text{ so} \\
\omega^T y &= [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [16 \ 25 \ 20 \ 4 \ 5 \ 1] \\
&= 1*16 + 1*25 + 1*20 + 1*4 + 1*5 + 1*1 \\
&= 71
\end{aligned}$$

Step 4:

$$\begin{aligned}
y_{21} = -y_{21} &= [-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1], \text{ so} \\
\omega^T y &= [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1] \\
&= 1 * (-4) + 1 * (-6.25) + 1 * (-5) + 1 * (-2) + 1 * (-2.5) + 1 * (-1) \\
&= -20.75
\end{aligned}$$

Step 5:

$$\begin{aligned}
y_{22} = -y_{22} &= [0 \ -4 \ 0 \ 0 \ -2 \ -1], \text{ so} \\
\omega^T y &= [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [0 \ -4 \ 0 \ 0 \ -2 \ -1] \\
&= 1 * 0 + 1 * (-4) + 1 * 0 + 1 * 0 + 1 * (-2) + 1 * (-1) \\
&= -7
\end{aligned}$$

Step 6:

$y_{23} = -y_{23} = [-4 \ -9 \ -6 \ -2 \ -3 \ -1]$, so

$$\omega^T y = [1 \ 1 \ 1 \ 1 \ 1 \ 1] * [-4 \ -9 \ -6 \ -2 \ -3 \ -1]$$

$$= 1 * (-4) + 1 * (-9) + 1 * (-6) + 1 * (-2) + 1 * (-3) + 1 * (-1)$$

$$= -25$$

Summary of the Steps:

y_{ij}	ω	$\omega^T y$
1 1 1 1 1 1	1 1 1 1 1 1	+6
1 1 -3 1 -3 1	1 1 1 1 1 1	-2
16 25 20 4 5 1	1 1 1 1 1 1	71
-4 -6.25 -5 -2 -2.5 -1	1 1 1 1 1 1	-20.75
0 -4 0 0 -2 -1	1 1 1 1 1 1	-7
-4 -9 -6 -2 -3 -1	1 1 1 1 1 1	-25

Before Iteration 2 the sum of those y_{ij} are take whose corresponding $\omega^T y$ are negative in the previous iteration.

Here,

$$\begin{aligned}\Sigma y_{ij} &= y_{12} + y_{21} + y_{22} + y_{23} \\ &= [1 \ 1 \ -3 \ 1 \ -3 \ 1] + [-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1] + \\ &\quad [0 \ -4 \ 0 \ 0 \ -2 \ -1] + [-4 \ -9 \ -6 \ -2 \ -3 \ -1] \\ &= [-7 \ -18.25 \ -14 \ -3 \ -10.5 \ -2]\end{aligned}$$

So the new ' ω ' is calculated as follows:

$$\begin{aligned}\omega &= \omega + \alpha * \Sigma y_{ij} \\ &= [1 \ 1 \ 1 \ 1 \ 1 \ 1] + 0.8 * [-7 \ -18.25 \ -14 \ -3 \ -10.5 \ -2] \\ &= [-4.6 \ -13.6 \ -10.2 \ -1.4 \ -7.4 \ -0.6]\end{aligned}$$

Repeat the iteration until for a particular ω the value of all $\omega^T y$ are positive or the number of iterations are finished.

Step 1:

$$y_{11} = [1 \ 1 \ 1 \ 1 \ 1 \ 1], \text{ so}$$

$$\begin{aligned}\omega^T y &= [-4.6 \ -13.6 \ -10.2 \ -1.4 \ -7.4 \ -0.6] * [1 \ 1 \ 1 \ 1 \ 1 \ 1] \\ &= (-4.6)*1 + (-13.6)*1 + (-10.2)*1 + (-1.4)*1 + (-7.4)*1 + (-0.6)*1 \\ &= -37.8\end{aligned}$$

Step 2:

$$y_{12} = [1 \ 1 \ -3 \ 1 \ -3 \ 1], \text{ so}$$

$$\omega^T y = [-4.6 \ -13.6 \ -10.2 \ -1.4 \ -7.4 \ -0.6] * [1 \ 1 \ -3 \ 1 \ -3 \ 1]$$

$$\begin{aligned}
&= (-4.6)*1 + (-13.6)*1 + (-10.2)*(-3) + (-1.4)*1 + (-7.4)*(-3) + (-0.6)*1 \\
&= 32.6
\end{aligned}$$

Step 3:

$$\begin{aligned}
Y_{13} &= [16 \ 25 \ 20 \ 4 \ 5 \ 1], \text{ so} \\
\omega^T y &= [-4.6 \ -13.6 \ -10.2 \ -1.4 \ -7.4 \ -0.6] * [16 \ 25 \ 20 \ 4 \ 5 \ 1] \\
&= (-4.6)*16 + (-13.6)*25 + (-10.2)*20 + (-1.4)*4 + (-7.4)*5 + (-0.6)*1 \\
&= -660.8
\end{aligned}$$

Step 4:

$$\begin{aligned}
y_{21} = -y_{21} &= [-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1], \text{ so} \\
\omega^T y &= [-4.6 \ -13.6 \ -10.2 \ -1.4 \ -7.4 \ -0.6] * \\
&[-4 \ -6.25 \ -5 \ -2 \ -2.5 \ -1] \\
&= (-4.6)*(-4) + (-13.6)*(-6.25) + (-10.2)*(-5) + (-1.4)*(-2) + (-7.4)*(-2.5) + (-0.6)*(-1) \\
&= 176.3
\end{aligned}$$

Step 5:

$$\begin{aligned}
Y_{22} = -Y_{22} &= [0 \ -4 \ 0 \ 0 \ -2 \ -1], \text{ so} \\
\omega^T y &= [-4.6 \ -13.6 \ -10.2 \ -1.4 \ -7.4 \ -0.6] * \\
&[0 \ -4 \ 0 \ 0 \ -2 \ -1]
\end{aligned}$$

$$= (-4.6)*0 + (-13.6)*(-4) + (-10.2)*0 + (-1.4)*0 + (-7.4)*(-2) + (-0.6)*(-1)$$

$$= 69.8$$

Step 6:

$$y_{23} = -y_{23} = [-4 \ -9 \ -6 \ -2 \ -3 \ -1] , \text{ so}$$

$$\omega^T y = [-4.6 \ -13.6 \ -10.2 \ -1.4 \ -7.4 \ -0.6] * [-4 \ -9 \ -6 \ -2 \ -3 \ -1]$$

$$= (-4.6)*(-4) + (-13.6)*(-9) + (-10.2)*(-6) + (-1.4)*(-2) + (-7.4)*(-3) + (-0.6)*(-1)$$

$$= 227.6$$

Summary of the Steps:

y_{ij}	ω	$\omega^T y$
1 1 1 1 1 1	-4.6 -13.6 -10.2 -1.4 -7.4 -0.6	-37.8
1 1 -3 1 -3 1	-4.6 -13.6 -10.2 -1.4 -7.4 -0.6	32.6
16 25 20 4 5 1	-4.6 -13.6 -10.2 -1.4 -7.4 -0.6	-660.8
-4 -6.25 -5 -2 -2.5 -1	-4.6 -13.6 -10.2 -1.4 -7.4 -0.6	176.3
0 -4 0 0 -2 -1	-4.6 -13.6 -10.2 -1.4 -7.4 -0.6	69.8
-4 -9 -6 -2 -3 -1	-4.6 -13.6 -10.2 -1.4 -7.4 -0.6	227.6

For the next iteration, we will get a new ω , which is calculated like the previous one. But since there are only TWO negative values of $\omega^T y$, so the new Σy_{ij} is calculated as :-

$$\Sigma y_{ij} = y_{11} + y_{13}$$

$$= [1 \ 1 \ 1 \ 1 \ 1 \ 1] + [16 \ 25 \ 20 \ 4 \ 5 \ 1]$$

$$= [17 \ 26 \ 21 \ 5 \ 6 \ 2]$$

So the new ' ω ' is calculated as follows:

$$\omega = \omega + \alpha * \Sigma y_{ij}$$

$$= [-4.6 \quad -13.6 \quad -10.2 \quad -1.4 \quad -7.4 \quad -0.6] + 0.8 * [17 \ 26 \ 21 \ 5 \ 6 \ 2]$$

$$= [9 \ 7.2 \ 6.6 \ 2.6 \ -2.6 \ 1]$$

Repeat the iteration until for a particular ω the value of all $\omega^T y$ are positive or the number of iterations are finished.